

# Securing Shared University Lab Computers

## Common Endpoint Vulnerabilities and a Free, Admin-Led Defense Architecture

Khondokar Sajid | ID: 2211954042 | Department of CSE, North South University  
Research direction under Dr. Nova Ahmed, DIAL Lab | Applicable to any institution with shared computing labs

### 1. Purpose and Scope

Shared computer laboratories at North South University — and at virtually every school, college, and university in Bangladesh — operate browsers and Windows in their default configuration. That default configuration silently exposes one user's credentials, personal data, and session state to the next person who sits down at the same machine. This document identifies the specific vulnerabilities present in a typical shared lab and specifies a complete, free, administrator-deployable defense for each. Every control listed is either built into Windows and Chrome/Edge or available as free open-source software. The total external software cost is zero.

The principle throughout is architectural, not behavioural: rather than training users not to make mistakes — an approach repeatedly shown ineffective in the security literature — each control removes the opportunity for the mistake to cause harm. The defense is designed to be deployed by an IT administrator on managed machines, measured before and after, and reproduced by any institution facing the same problem.

### 2. Deployment Principle and Rollback

All browser and Windows policies in this document deploy through Group Policy (using free Chrome/Edge ADMX templates) or Chrome Browser Cloud Management (free, unlimited managed browsers), and every one is reversible within minutes — deleting the relevant policy reverts the change at the next user login. Execution-control policies (AppLocker/WDAC) and Defender Attack Surface Reduction rules should first be deployed in Audit mode for one to two weeks to identify legitimate software that would be wrongly blocked, then switched to Block mode after exceptions are established. The entire architecture should be piloted on a single laboratory and measured before any campus-wide rollout.

*Reversibility is the key reassurance for any IT office: nothing here is permanent, and a misconfiguration can be undone faster than it was applied.*

### 3. Vulnerabilities in the Shared Lab Environment

The following vulnerabilities are present by default on shared lab machines. None requires the attacker to deploy malware or exploit a software defect; most exploit the intended, documented behaviour of the browser and operating system.

| #  | Vulnerability                          | How it is exploited                                                                                                |
|----|----------------------------------------|--------------------------------------------------------------------------------------------------------------------|
| V1 | Sync-persistent profile leak           | A user signs into Chrome with a personal account, enables sync, and logs out of email but not the browser. Pass    |
| V2 | Ghost profile residue                  | Even after browser sign-out, the local profile folder remains on disk with cookies, session tokens, form data, and |
| V3 | Software keylogger                     | A student runs a hidden Python script or compiled executable that captures every keystroke of subsequent users     |
| V4 | Malicious browser extension            | An installed extension (a fake PDF viewer, a 'Tackker'-type tool) logs keystrokes, captures screenshots, and rea   |
| V5 | Incognito inadequacy                   | Users believe incognito mode protects them. It does not stop an already-installed keylogger, a malicious extens    |
| V6 | Hardware USB keylogger                 | A physical device placed between the keyboard and the PC records all input. It is invisible to all software-based  |
| V7 | Cross-device sync bleed (BYOD)         | A student with sync enabled on a personal device pulls personal photos, passwords, and history onto a lab mach     |
| V8 | No existing free institutional defense | No documented, free, reproducible institutional solution exists for Bangladeshi universities. Commercial tools a   |

### 4. Proposed Solutions — Admin-Led Lockdown

Each solution below maps to one or more vulnerabilities above. For each, the exact setting and the operational trade-off (what breaks, who is affected) are stated, because an institution must understand the cost before deploying.

#### 4.1 — Force Ephemeral Browser Profiles (addresses V1, V2)

Setting: Chrome/Edge ADMX → *Force ephemeral profile* → Enabled. Every session runs in a fresh temporary profile that is deleted when the browser closes.

*Trade-off: Students lose all locally saved bookmarks, history, and passwords each session. They must use cloud storage (Google Drive) or email work to themselves. Communicate this before deployment.*

#### 4.2 — Block Browser Sync Completely (addresses V1, V7)

Setting: Chrome ADMX → *Enable Sync* → Disabled (or legacy *SyncDisabled* = *true*). Also set *BrowserSignin* = 0 so the browser cannot sign into any Google account at all. Edge: disable *Configure sync*.

*Trade-off: Students cannot use personal Chrome sync to access their own bookmarks or passwords on lab PCs. This is the intended outcome on a shared machine, but users must be told.*

#### 4.3 — Disable Credential Storage and Form Fill (addresses V2)

Setting: Chrome/Edge → *Enable saving passwords to the password manager* → Disabled. Also set *AutofillAddressEnabled* and *AutofillCreditCardEnabled* to Disabled.

*Trade-off: No more save-password prompts; mild inconvenience for autofill on non-sensitive forms.*

#### 4.4 — Block Extensions, Allow Only an Approved List (addresses V4)

Setting: *ExtensionInstallBlocklist* = [""], then *ExtensionInstallAllowlist* with vetted extension IDs only. Set *BlockExternalExtensions* = 1 and *DeveloperToolsAvailability* = 2 (disabled). Edge: equivalent policies.

*Trade-off: Legitimate extensions (Grammarly, citation managers, accessibility tools) are blocked unless explicitly allowlisted. CSE labs that need DevTools require a separate, looser policy applied only to those machines.*

#### 4.5 — Neutralize Software Keyloggers (addresses V3)

Settings, applied together:

- Remove local administrator rights — all student accounts run as standard users. This is the foundation; execution controls are far weaker if a student is a local admin who can disable them.
- Enable AppLocker or Windows Defender Application Control (WDAC) in whitelist mode — only signed/approved executables run; arbitrary Python scripts and downloaded keyloggers are blocked, especially from *C:\Users\\** and removable drives.
- Apply PowerShell Constrained Language Mode to break reflection-based malicious scripts.
- Keep Windows Defender Antivirus with cloud-delivered protection and PUA blocking enabled.
- Enable the Defender ASR rule *Block credential stealing from LSASS* (GUID 9e6c4e1f-7d60-472f-ba1a-a39ef669e4b2).

Note: this defends against credential dumping, which is distinct from keylogging — both are needed.

*Trade-off: Any student project requiring an unapproved executable, Python script, or USB tool will fail. IT must maintain an allowlist and handle exception requests — a real administrative load. Best applied strictly to general labs; CSE labs may need a looser policy backed by stronger detection (Section 4.9).*

#### 4.6 — Prevent Hardware USB Keyloggers (addresses V6)

Setting: Device Installation Restrictions GPO — *Prevent installation of devices not described by other policy settings*, then allowlist only the known keyboard/mouse hardware IDs. Physically lock USB ports in high-risk rooms; prefer machines with integrated keyboards where possible.

*Trade-off: Students cannot use their own USB keyboards or mice. Honest limitation: no software fully defeats a hardware keylogger placed inline — physical inspection and port control are the only real defenses, and they reduce rather than eliminate the risk.*

#### 4.7 — Non-Persistent Lab Machines: Windows UWF or Mandatory Profiles (addresses V2, V3, V4)

Setting: Enable the Unified Write Filter (UWF) on Windows 10/11 Education/Enterprise. All writes are redirected to a virtual overlay and discarded on reboot, returning the machine to a pristine state. UWF is the free equivalent of the commercial Deep Freeze. Mandatory (read-only) user profiles are an alternative.

*Trade-off: Anything installed or saved to the system drive after boot is lost on restart; updates require a maintenance window with UWF temporarily disabled. Students must save all work to a network drive or cloud. This is the strongest single reset mechanism for open labs.*

**Important interaction: UWF and Windows Shared PC Mode both reset machine state and should not be stacked blindly. Choose one primary reset mechanism per machine — UWF for machines that reboot frequently, Shared PC Mode (delete-profile-on-logout) for machines that stay on all day. State which was chosen and why.**

#### 4.8 — Cross-Device Sync Block for BYOD (addresses V7)

Approach: Lab-side ephemeral profiles (4.1) and sign-in blocking (4.2) already prevent synced data from arriving on lab machines. For personal devices, if the institution's identity platform supports context-aware access (e.g., Google Workspace

Context-Aware Access), unmanaged browsers can be blocked from university services. Otherwise, provide students a short guide to disable sync for the university account on their personal devices.

*Trade-off: If context-aware access is enforced, some students on older devices or browsers may face login issues and need testing. Note that most institutions on free Google Workspace for Education Fundamentals do not have context-aware access, so the guide approach is the realistic fallback.*

#### 4.9 — Detection Layer: Sysmon + Wazuh (catches what prevention misses)

Setting: Deploy Microsoft Sysmon with a curated configuration on lab machines; forward logs to a free Wazuh server (an old PC suffices for one lab). Alert on:

- The Chrome *Login Data* file being copied out of its profile directory.
- Suspicious process creation — *pythonw.exe*, hidden *cmd.exe* or PowerShell.
- LSASS memory access (credential dumping).
- Persistence via registry Run keys or WMI event subscription.
- Outbound connections to Telegram or Discord APIs from non-browser processes (exfiltration).

*Trade-off: Requires a small server and an IT member able to tune alerts; initial alert noise must be calibrated. This is not set-and-forget — it is the necessary visibility for when prevention fails.*

**Privacy and trust: monitoring covers process and network metadata only — not document contents, keystrokes within sessions, or browsing activity. A published monitoring policy should state this plainly so the detection layer protects students without surveilling them.**

### 5. Vulnerability-to-Solution Map

| Vulnerability              | Primary solution(s)                                   | Fully solved?                                      |
|----------------------------|-------------------------------------------------------|----------------------------------------------------|
| V1 Sync-persistent profile | 4.1 Ephemeral profiles, 4.2 Block sync                | Yes                                                |
| V2 Ghost profile residue   | 4.1, 4.3, 4.7 UWF / mandatory profiles                | Yes                                                |
| V3 Software keylogger      | 4.5 Standard user + AppLocker, 4.7 UWF, 4.9 Detection | Yes (with standard-user accounts)                  |
| V4 Malicious extension     | 4.4 Extension blocklist, 4.7 UWF                      | Yes                                                |
| V5 Incognito inadequacy    | Solved by fixing V3 and V4                            | Yes (indirectly)                                   |
| V6 Hardware keylogger      | 4.6 USB control + physical inspection                 | No &mdash; reduced only; software cannot defeat it |
| V7 Cross-device sync bleed | 4.1, 4.2 lab-side; 4.8 BYOD guide                     | Lab side yes; BYOD partial                         |
| V8 No free defense exists  | This entire architecture                              | Yes &mdash; that is the contribution               |

### 6. The Minimal Effective Subset

An institution does not need all nine controls on day one. The research and the deployment are both stronger if the focus is the smallest set that closes the most attack surface for the least breakage. The recommended minimal backbone is:

- **Standard-user accounts** (removes attacker privilege — the foundation under everything else).
- **One reset mechanism** — UWF or Shared PC Mode (destroys all residual data each cycle).
- **Force ephemeral profiles + block sign-in/sync** (kills the browser leak directly).
- **Sysmon + Wazuh** (sees anything that slips through).

*These four close V1, V2, V3, V4, V5, and V7 almost entirely. The remaining controls (extension allowlist detail, USB hardware control, BYOD guidance) are refinements layered on afterward. Deploying the minimal set first, measuring it, then adding refinements is both better engineering and a cleaner research design.*

### 7. How to Prove It Works — Before and After

The contribution is not the list of controls; it is the measured evidence that they close the attack surface. Deploy on one lab (Lab A); keep a matched lab unchanged as control (Lab B). For each machine, before and after deployment, attempt credential extraction using only researcher-owned test accounts — never real student data. Record per machine whether extraction succeeded. The expected result: Lab A moves from mostly-extractable to non-extractable; Lab B is unchanged. Pair this with an anonymous bilingual survey on user behaviour and awareness, before and after, to capture the human dimension.

| Metric | Lab A Before | Lab A After | Lab B (control) |
|--------|--------------|-------------|-----------------|
|--------|--------------|-------------|-----------------|

|                                      |                |                    |           |
|--------------------------------------|----------------|--------------------|-----------|
| Credential extraction success rate   | high (measure) | near zero (target) | unchanged |
| Machines with sync/sign-in disabled  | measure        | 100%               | unchanged |
| Machines resetting state each cycle  | measure        | 100%               | unchanged |
| Survey: % unaware of the risk        | measure        | measure            | n/a       |
| Detection: suspicious events flagged | n/a            | record             | n/a       |

## 8. Applicability to Other Institutions

Nothing in this architecture is specific to NSU. Every control is a standard, documented feature of Windows, Chrome, Edge, and free open-source tools. Any school, college, or university running shared Windows lab machines with Chrome or Edge faces the same default vulnerabilities and can deploy the same defense at zero license cost. Publishing the architecture, the deployment scripts, and the before-and-after measurement openly allows other Bangladeshi institutions — which face an identical and largely unaddressed problem — to reproduce it directly. This reproducibility is the practical value of the work beyond NSU.

---

*Every control listed removes an opportunity for harm rather than asking a user to behave differently. No single layer is complete: standard-user accounts and a reset mechanism form the backbone, the browser policies close the direct leak, and detection covers the rest. Hardware keyloggers remain the one threat that software cannot defeat — stated honestly. The architecture is free, reversible, reproducible, and ready to pilot on a single lab.*

Khondokar Sajid | ID: 2211954042 | CSE, North South University | DIAL Lab